

DOSSIER PROJET

Titre Professionnel

Développeur Web et Web Mobile (DWWM)

Candidat : Amad-Khaly Diallo

Centre de formation : La Plateforme, Paris

Niveau : RNCP Niveau 5 — Bac+2

Durée de formation : 16 mois

Projets présentés

marsAI — Plateforme full-stack de festival de films courts

artisan-ai — Assistant WhatsApp IA pour artisans (SaaS)

Juin 2026

Table des matières

(Mettre à jour la table : clic droit → Mettre à jour les champs)

1. Introduction

1.1 Présentation du candidat

Je m'appelle Amad-Khaly Diallo. Après un parcours orienté vers le numérique, j'ai intégré la formation Développeur Web et Web Mobile (DWWM) dispensée par La Plateforme à Paris, un centre de formation spécialisé dans les métiers du digital. Cette formation intensive de 16 mois m'a permis d'acquérir les compétences techniques nécessaires à la conception et au développement d'applications web complètes, aussi bien côté front-end que côté back-end.

Ce dossier constitue la synthèse de mon parcours de formation. Il présente deux projets que j'ai réalisés, démontrant ma maîtrise des activités types définies par le référentiel DWWM : le développement de la partie front-end d'une application web sécurisée (AT1) et le développement de la partie back-end d'une application web sécurisée (AT2).

1.2 Contexte et parcours de formation

La formation DWWM de La Plateforme est structurée autour d'une pédagogie par projets. Durant 16 mois, j'ai progressivement maîtrisé l'ensemble du cycle de développement web : de la conception d'interfaces utilisateur à la mise en place d'architectures back-end robustes, en passant par la gestion de bases de données, la sécurisation des applications et le travail en équipe avec des méthodes agiles.

Les compétences acquises couvrent un spectre technologique large et représentatif des besoins actuels du marché :

- Langages : HTML5, CSS3, JavaScript (ES6+), TypeScript
- Frameworks front-end : React 19, Next.js 14 (App Router)
- Frameworks back-end : Node.js, Express.js
- Bases de données : MariaDB / MySQL, PostgreSQL (via Supabase)
- Outils et services : Git, GitHub, Docker, Sanity CMS, Scaleway S3, Stripe, Twilio, API Claude (Anthropic)
- Sécurité : JWT, bcrypt, Helmet, CORS, rate limiting, validation des signatures HMAC
- Méthodologie : Agile, stand-ups quotidiens, GitHub Projects

Tout au long de cette formation, j'ai développé la capacité à rechercher et exploiter des ressources techniques en anglais — langue dominante de la documentation officielle — ce qui est indispensable dans le métier de développeur.

1.3 Présentation des projets retenus

Ce dossier s'appuie sur deux projets réels que j'ai développés au cours de ma formation.

Projet 1 — marsAI

marsAI est une plateforme web full-stack dédiée à la gestion d'un festival de films courts intégrant l'intelligence artificielle. Ce projet a été réalisé en équipe de cinq développeurs, dont j'ai occupé le rôle de lead développeur, avec la responsabilité exclusive du back-end et une contribution partielle au front-end. La plateforme permet aux réalisateurs de soumettre leurs films, au jury de les évaluer, et à l'équipe organisatrice de gérer l'ensemble du festival via un tableau de bord d'administration sécurisé.

Stack technique : React 19, Node.js/Express, MariaDB, Docker, Sanity CMS, Scaleway S3, Brevo, YouTube API.

Projet 2 — artisan-ai

artisan-ai est un SaaS (Software as a Service) que j'ai développé en solo. Il permet aux artisans de recevoir et répondre automatiquement aux messages WhatsApp de leurs clients grâce à l'intelligence artificielle (Claude d'Anthropic). L'artisan dispose d'un tableau de bord web pour consulter ses conversations, gérer son profil et souscrire à un abonnement via Stripe.

Stack technique : Next.js 14 (App Router), TypeScript, Supabase, Stripe, Twilio, API Claude.

2. Projet marsAI — Plateforme de festival de films courts

2.1 Cahier des charges et expression des besoins

2.1.1 Contexte et objectif

L'objectif de marsAI était de concevoir une plateforme web complète pour un festival de films courts. Le client avait besoin d'un outil permettant de :

- Présenter le festival (informations, programme, jury, partenaires)
- Permettre aux réalisateurs de soumettre leurs films via un formulaire multi-étapes
- Offrir à un jury sélectionné d'évaluer les films soumis via un espace dédié
- Donner à l'équipe organisatrice un tableau de bord d'administration complet
- Gérer une newsletter et une liste d'abonnés
- Afficher la galerie des films sélectionnés et les gagnants
- S'adapter à trois phases temporelles du festival (avant, pendant, après)

2.1.2 Utilisateurs cibles

La plateforme devait servir trois types d'utilisateurs distincts :

- Les visiteurs publics : accès aux informations du festival, galerie, formulaire de contact, newsletter
- Les réalisateurs participants : soumission de films via un formulaire multi-étapes guidé
- Les administrateurs et membres du jury : accès à un back-office sécurisé pour la gestion complète du festival

2.1.3 Contraintes techniques

- Application responsive (desktop et mobile)
- Interface multilingue (français, anglais, arabe)
- Stockage des médias sur cloud (Scaleway S3, compatible AWS)
- Intégration de la chaîne YouTube du festival pour la diffusion des films
- Envoi d'emails transactionnels et marketing via Brevo
- Contenu éditorial géré via un CMS headless (Sanity)
- Sécurité renforcée : authentification JWT, protection contre les attaques courantes

[CAPTURE : Capture du cahier des charges ou d'un tableau Kanban GitHub Projects montrant les user stories]

2.2 Gestion de projet

2.2.1 Méthodologie Agile

Pour marsAI, l'équipe de cinq développeurs a adopté une méthodologie Agile adaptée au contexte de formation. Nous avons organisé notre travail en sprints d'une à deux semaines, avec des rituels réguliers :

- Daily stand-ups : réunions quotidiennes courtes (15 minutes) pour synchroniser l'avancement, identifier les blocages et ajuster les priorités
- Revues de sprint : démonstration des fonctionnalités terminées en fin de sprint
- GitHub Projects : utilisation du tableau Kanban intégré à GitHub pour suivre les tâches (colonnes To Do / In Progress / Review / Done)
- Pull Requests et code reviews : chaque fonctionnalité était développée sur une branche dédiée, soumise en PR et revue par un autre membre avant merge

2.2.2 Gestion des versions avec Git

Nous avons utilisé Git avec une convention de branches inspirée de Git Flow :

- main : branche de production, protégée
- develop : branche d'intégration
- feature/[nom] : branches de fonctionnalités individuelles
- fix/[nom] : branches de correctifs

Les messages de commit suivaient la convention Conventional Commits (feat:, fix:, refactor:, docs:, etc.) pour maintenir un historique lisible.

[CAPTURE : Capture du tableau GitHub Projects avec les colonnes Kanban et les issues]

[CAPTURE : Capture du graphe de branches Git ou de la liste des Pull Requests]

2.2.3 Répartition des rôles

En tant que lead développeur, j'ai eu la responsabilité de :

- Concevoir et implémenter l'intégralité du back-end (API Express, base de données, services tiers)
- Définir l'architecture technique globale du projet
- Mettre en place les standards de code et les conventions de l'équipe
- Réviser les Pull Requests des autres membres
- Coordonner l'intégration entre le front-end et le back-end

2.3 Spécifications techniques

2.3.1 Architecture générale

marsAI est une application web full-stack organisée en trois couches distinctes :

- Front-end : Single Page Application (SPA) React 19, déployée indépendamment
- Back-end : API REST Express.js, point d'entrée unique pour toutes les données dynamiques
- CMS : Sanity headless CMS, gérant le contenu éditorial (textes, images, informations du festival)

Flux de données principal :

```
Browser → React Router → Page Component
  → services/api.js (appels Axios)
  → Express Routes (/api/*)
    → Middlewares (auth JWT, rate-limit, phase-check)
      → Controllers (couche mince)
        → Services (logique métier + requêtes SQL)
          → MariaDB / Scaleway S3 / YouTube API / Brevo
```

2.3.2 Justification des choix technologiques

React 19 (front-end)

React a été choisi pour sa popularité dans l'industrie, son écosystème riche et la facilité qu'il offre pour construire des interfaces dynamiques et réactives. La version 19, utilisée ici, apporte des améliorations de performance et de nouveaux hooks. La bibliothèque Tailwind CSS a été associée à React pour une stylisation rapide et cohérente, basée sur des classes utilitaires.

Node.js / Express.js (back-end)

Node.js permet d'utiliser JavaScript côté serveur, ce qui réduit la friction entre front et back pour l'équipe. Express.js est un framework minimaliste et flexible, idéal pour construire des API REST robustes. Son écosystème de middlewares (Helmet, cors, express-rate-limit) facilite la mise en place des couches de sécurité.

MariaDB (base de données)

MariaDB est un système de gestion de base de données relationnelle open source, fork de MySQL, réputé pour ses performances et sa conformité SQL. Il a été lancé via Docker pour garantir la reproductibilité de l'environnement de développement sur toutes les machines de l'équipe.

Sanity CMS (gestion de contenu)

Sanity est un CMS headless qui permet à l'équipe non-technique de modifier le contenu du site (textes, images, membres du jury, partenaires) sans toucher au code. Les données sont exposées via une API GraphQL/GROQ que le front-end consomme directement.

Scaleway S3 (stockage de médias)

Scaleway propose un service de stockage objet compatible avec l'API Amazon S3. Les films soumis et les images associées sont stockés sur ce service cloud plutôt que sur le serveur, ce qui garantit la scalabilité et soulage la bande passante du serveur applicatif.

Brevo (emails transactionnels et marketing)

Brevo (anciennement Sendinblue) est un service d'emailing qui permet l'envoi d'emails transactionnels (confirmations de soumission) et la gestion de la newsletter du festival. Il expose une API REST simple et un serveur SMTP.

[CAPTURE : Schéma d'architecture global de marsAI (3 couches : front / back / services tiers)]

2.4 AT1 — Développement de la partie front-end

2.4.1 Maquettage et charte graphique

Bien que ce projet n'ait pas bénéficié de maquettes Figma formalisées, l'équipe a défini dès le départ une charte graphique cohérente inspirée de l'univers cinématographique :

- Palette de couleurs : tons sombres (noir, gris anthracite) avec des accents dorés/ambrés pour évoquer le cinéma et les projecteurs
- Typographie : Calibri pour les corps de texte, polices sans-serif pour les titres
- Composants : boutons arrondis, cartes avec ombre portée, animations fluides pour les transitions
- Responsive : grille mobile-first avec Tailwind CSS (sm / md / lg / xl breakpoints)

La structure des pages principales a été esquissée informellement lors des réunions d'équipe avant d'être implémentée directement en code.

[CAPTURE : Screenshots de la page d'accueil (phase 1, 2 et 3 du festival)]

[CAPTURE : Screenshots du formulaire de soumission multi-étapes (Participer)]

[CAPTURE : Screenshot du tableau de bord administrateur]

2.4.2 Interfaces statiques et organisation des composants

Le front-end est organisé selon une architecture de composants React claire et modulaire :

- src/pages/ : composants de niveau page (Home, Participer, Contact, Gallery, Winners, Admin...)
- src/components/ : composants réutilisables organisés par domaine (admin/, home/, participer/, forms/, layout/, ui/)
- src/router/ : configuration du routage avec React Router
- src/services/ : couche d'abstraction pour les appels API
- src/contexts/ : état global via Context API (AuthContext, AdminContext)
- src/hooks/ : hooks personnalisés (useAuth, useSubmission, useFestivalPhase...)

- `src/i18n/` : fichiers de traduction JSON (fr, en, ar)

Exemple de composant réutilisable — le système de phases festival. Le composant `HomePhase2` affiche la page d'accueil pendant la période de soumission des films, avec des statistiques en temps réel et une section programme :

```
// src/components/home/Phase2/HomePhase2.jsx
// Affichage conditionnel selon la phase active du festival
import HeroCamera from './HeroCamera';
import StatsSection from './StatsSection';
import ProgramSection from './ProgramSection';
import ManifestoSection from './ManifestoSection';

export default function HomePhase2() {
  return (
    <main>
      {/* Hero avec caméra animée — symbolise la phase de tournage */}
      <HeroCamera />
      {/* Statistiques live : films soumis, pays, etc. */}
      <StatsSection />
      {/* Programme et agenda du festival */}
      <ProgramSection />
      {/* Manifeste de l'IA dans le cinéma */}
      <ManifestoSection />
    </main>
  );
}
```

2.4.3 Interfaces dynamiques

Système de phases du festival

L'une des fonctionnalités les plus importantes du front-end est le système de phases. La plateforme adapte dynamiquement son contenu selon la phase active (Phase 1 : annonce, Phase 2 : soumission, Phase 3 : post-festival). Ce comportement est piloté par un hook personnalisé qui interroge le back-end :

```
// src/hooks/useFestivalPhase.js
// Récupère la phase active depuis l'API et l'expose à toute l'app
import { useState, useEffect } from 'react';
import api from '../services/api';

export default function useFestivalPhase() {
  const [phase, setPhase] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    api.get('/festival-phase')
      .then(res => setPhase(res.data.phase))
      .catch(() => setPhase('phase1')) // Fallback sûr
      .finally(() => setLoading(false));
  }, []);
}
```

```
return { phase, loading };
}
```

Formulaire de soumission multi-étapes

Le formulaire de soumission de films (page Participer) est une fonctionnalité complexe décomposée en plusieurs étapes guidées. Chaque étape est un composant indépendant géré par un stepper central :

- Étape 1 : Informations sur le réalisateur (FilmmakerStep)
- Étape 2 : Informations sur le film (MovieStep)
- Étape 3 : Tags et assets (AssetsTagsStep)
- Étape 4 : Collaborateurs (CollaboratorsStep)
- Étape 5 : Déclaration d'utilisation de l'IA (AIDeclarationStep)

```
// src/components/participer/SubmissionStepper.jsx (simplifié)
// Gère la navigation entre les étapes du formulaire de soumission
const STEPS = [
  { label: 'Réalisateur',    component: FilmmakerStep },
  { label: 'Film',          component: MovieStep },
  { label: 'Assets & Tags',  component: AssetsTagsStep },
  { label: 'Collaborateurs', component: CollaboratorsStep },
  { label: 'Déclaration IA', component: AIDeclarationStep },
];

export default function SubmissionStepper() {
  const [currentStep, setCurrentStep] = useState(0);
  const CurrentComponent = STEPS[currentStep].component;

  return (
    <div>
      { /* Indicateur de progression */ }
      <StepIndicator steps={STEPS} current={currentStep} />
      { /* Étape active */ }
      <CurrentComponent
        onNext={() => setCurrentStep(s => s + 1)}
        onPrev={() => setCurrentStep(s => s - 1)}
      />
    </div>
  );
}
```

Internationalisation (i18n)

La plateforme est disponible en trois langues grâce à la bibliothèque i18next. La détection de langue est automatique (basée sur le navigateur) et l'utilisateur peut changer manuellement via le sélecteur dans le header. Les traductions sont stockées dans des fichiers JSON par langue :

```
// src/i18n.js - Configuration de l'internationalisation
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
import frTranslation from './i18n/fr/translation.json';
```

```
import enTranslation from './i18n/en/translation.json';

i18n.use(initReactI18next).init({
  resources: {
    fr: { translation: frTranslation },
    en: { translation: enTranslation },
  },
  lng: 'fr', // Langue par défaut
  fallbackLng: 'fr', // Langue de secours si une clé est manquante
  interpolation: { escapeValue: false }, // React échappe déjà le HTML
});

export default i18n;
```

2.4.4 Démarche de sécurité côté front-end

Protection XSS (Cross-Site Scripting)

React protège nativement contre les attaques XSS en échappant automatiquement toutes les valeurs insérées dans le JSX. Toutes les données utilisateur sont affichées via des expressions JSX standard (`{variable}`) et jamais via `dangerouslySetInnerHTML`, ce qui élimine le vecteur d'injection HTML/JavaScript.

Validation des formulaires côté client

Tous les formulaires sont validés côté client avant l'envoi, avec des messages d'erreur clairs pour guider l'utilisateur. Cette validation côté client est complémentaire (et non substituée) à la validation côté serveur :

```
// src/utils/validation.js (extrait)
// Validation du formulaire de soumission de film
export function validateMovieForm(data) {
  const errors = {};

  if (!data.title || data.title.trim().length < 2)
    errors.title = 'Le titre doit comporter au moins 2 caractères';

  if (!data.duration || isNaN(data.duration) || data.duration <= 0)
    errors.duration = 'La durée doit être un nombre positif';

  if (!data.year || data.year < 2020 || data.year > new Date().getFullYear())
    errors.year = 'Année de production invalide';

  return errors; // Objet vide = formulaire valide
}
```

Gestion sécurisée de l'authentification

L'authentification côté client est gérée via l'AuthContext. Le token JWT n'est jamais stocké dans localStorage (vulnérable aux attaques XSS) mais dans un cookie HTTP-only géré par le serveur. Le front-end se contente de vérifier l'état d'authentification via un appel API :

```
// src/contexts/AuthContext.jsx (extrait)
// Le token JWT est stocké en cookie HTTP-only côté serveur
// Le client vérifie son état via un endpoint sécurisé
const checkAuth = async () => {
  setChecking(true);
  try {
    const ok = await adminService.checkAuth(); // Appel GET /api/admins/auth/me
    setIsAuthenticated(!ok);
    return ok;
  } catch (err) {
    setIsAuthenticated(false); // En cas d'erreur → non authentifié
    return false;
  } finally {
    setChecking(false);
  }
};
```

[CAPTURE : Screenshot de la page d'administration avec le système de rôles (admin / super_admin)]

2.5 AT2 — Développement de la partie back-end

2.5.1 Conception de la base de données

Modèle Conceptuel de Données (MCD)

La base de données marsAI organise les données autour de plusieurs entités principales :

- admins : les membres de l'équipe d'organisation avec leurs rôles (admin / super_admin)
- movies : les films soumis par les réalisateurs, avec leurs statuts et métadonnées
- filmmakers : les profils des réalisateurs, liés à leurs films
- collaborators : les collaborateurs d'un film (co-réalisateurs, acteurs, etc.)
- jury : les membres du jury
- partners : les partenaires du festival
- newsletters : les abonnés à la newsletter
- festival_phases : la phase temporelle active du festival
- admin_movie_assignment : table de liaison entre admins et films (pour l'évaluation : notes, flags)

[CAPTURE : Schéma MCD/MLD de la base de données marsAI (entités et relations)]

Schéma de la table principale movies

```
-- Structure de la table movies (extrait du dump marsAi.sql)
CREATE TABLE `movies` (
  `id` int(11) UNSIGNED NOT NULL AUTO_INCREMENT,
  `title` varchar(255) NOT NULL,
  `duration` int(11) DEFAULT NULL, -- en secondes
  `year` year(4) DEFAULT NULL,
  `country` varchar(100) DEFAULT NULL,
  `synopsis` text DEFAULT NULL,
  `youtube_url` varchar(500) DEFAULT NULL,
  `youtube_status` enum('pending','published','unlisted') DEFAULT 'pending',
```

```

`status`          enum('pending','approved','rejected') DEFAULT 'pending',
`ai_used`         tinyint(1) DEFAULT 0,          -- déclaration d'utilisation
IA
`submitted_at`    timestamp DEFAULT current_timestamp(),
`updated_at`      timestamp DEFAULT current_timestamp() ON UPDATE
current_timestamp(),
PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Table d'évaluation des films par les admins

```

-- Table de liaison admin <-> film (notation par le jury)
CREATE TABLE `admin_movie_assignment` (
  `id`          int(11) NOT NULL AUTO_INCREMENT,
  `admin_id`    int(11) NOT NULL,
  `movie_id`    int(11) NOT NULL,
  `rating`      tinyint(4) DEFAULT NULL,  -- note de 1 à 10
  `comment`     text DEFAULT NULL,
  `flag`        enum('green','yellow','red') DEFAULT NULL, --
sélection/réserve/rejet
  `created_at`  datetime NOT NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id`),
  FOREIGN KEY (`admin_id`) REFERENCES `admins` (`id`),
  FOREIGN KEY (`movie_id`) REFERENCES `movies` (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Table des admins avec hachage des mots de passe

```

-- Structure de la table admins
CREATE TABLE `admins` (
  `id`          int(10) UNSIGNED NOT NULL AUTO_INCREMENT,
  `first_name`  varchar(100) NOT NULL,
  `last_name`   varchar(100) NOT NULL,
  `email`       varchar(150) NOT NULL UNIQUE,
  `password`    varchar(255) NOT NULL,  -- hash bcrypt (coût 10)
  `role`        enum('admin','super_admin') NOT NULL DEFAULT 'admin',
  `created_at`  timestamp DEFAULT current_timestamp(),
  `updated_at`  timestamp DEFAULT current_timestamp() ON UPDATE
current_timestamp(),
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

2.5.2 Composants d'accès aux données

Le back-end suit une architecture MVC (Modèle-Vue-Contrôleur) adaptée aux API REST, avec une séparation claire des responsabilités :

Configuration du pool de connexions MariaDB

La connexion à la base de données utilise un pool de connexions pour optimiser les performances et éviter l'épuisement des connexions lors de pics de trafic :

```

// backend/Config/db.js
// Pool de connexions MariaDB - évite d'ouvrir une connexion par requête

```

```

const mysql = require('mysql2/promise');
const env = require('./env');

const pool = mysql.createPool({
  host:          env.DB_HOST,
  port:          env.DB_PORT,
  database:      env.DB_NAME,
  user:          env.DB_USER,
  password:      env.DB_PASSWORD,
  waitForConnections: true,
  connectionLimit: 10, // Max 10 connexions simultanées
  queueLimit:    0, // File d'attente illimitée
});

async function checkDatabaseConnection() {
  const conn = await pool.getConnection();
  await conn.ping();
  conn.release();
  console.log('Connexion à la base de données établie.');
```

```

}

module.exports = { pool, checkDatabaseConnection };

```

Structure Controllers / Services

Les contrôleurs sont des couches minces qui délèguent la logique aux services. Les services contiennent les requêtes SQL et la logique métier :

```

// Pattern contrôleur / service - exemple pour les films

// backend/Controllers/movieController.js
const movieService = require('../Services/movieService');

exports.getMovies = async (req, res, next) => {
  try {
    const movies = await movieService.findAll(req.query);
    res.json({ success: true, data: movies });
  } catch (err) {
    next(err); // Délègue à errorHandler
  }
};

// backend/Services/movieService.js
const { pool } = require('../Config/db');

exports.findAll = async ({ status, page = 1, limit = 20 }) => {
  // Requête paramétrée - protection contre l'injection SQL
  const [rows] = await pool.query(
    'SELECT m.*, f.name AS filmmaker_name '
    + 'FROM movies m '
    + 'LEFT JOIN filmmakers f ON f.movie_id = m.id '
    + 'WHERE (? IS NULL OR m.status = ?) '
    + 'ORDER BY m.submitted_at DESC '
    + 'LIMIT ? OFFSET ?',
    [status ?? null, status ?? null, parseInt(limit), (page - 1) *

```

```

parseInt(limit)]
);
return rows;
};

```

2.5.3 API REST Express — Routes et middlewares

L'API expose une douzaine de ressources, toutes montées sous le préfixe /api. Voici les principales :

Route	Méthode	Accès	Description
/api/movies	GET	Public	Liste des films
/api/movies/submit	POST	Public	Soumission d'un film
/api/admin/films	GET	Admin	Films + statuts (admin)
/api/admin/films/:id/flag	PATCH	Admin	Flagging vert/jaune/rouge
/api/admins/auth/login	POST	Public	Connexion admin
/api/admins/auth/logout	POST	Admin	Déconnexion
/api/jury	GET	Public	Liste du jury
/api/partners	GET	Public	Partenaires
/api/newsletters/subscribe	POST	Public	Inscription newsletter
/api/festival-phase	GET	Public	Phase active du festival
/api/upload/video	POST	Public	Upload vidéo → S3
/api/contact	POST	Public	Formulaire de contact

2.5.4 Démarche de sécurité côté back-end

Headers de sécurité HTTP avec Helmet

Helmet est un middleware Express qui configure automatiquement une série de headers HTTP de sécurité, réduisant la surface d'attaque de l'application :

```

// backend/server.js - Configuration Helmet
const helmet = require('helmet');

app.use (
  helmet({
    // crossOriginResourcePolicy: permet aux assets publics
    // (images, vidéos) d'être chargés depuis d'autres origines
    crossOriginResourcePolicy: { policy: 'cross-origin' },
  })
);
// Helmet positionne automatiquement :
// - X-Frame-Options: DENY (protection clickjacking)
// - X-Content-Type-Options: nosniff (protection MIME sniffing)
// - Strict-Transport-Security (HTTPS forcé)
// - X-XSS-Protection: 0 (désactivé car géré par CSP)
// - Content-Security-Policy (CSP)

```

CORS — Whitelist des origines autorisées

```

// backend/server.js - CORS strict en production
const allowedOrigins = (process.env.FRONTEND_URL || 'http://localhost:3000')
  .split(',').map(o => o.trim());

```

```

app.use(cors({
  origin: (origin, callback) => {
    // En développement : autorise les requêtes sans origin (Postman, curl)
    if (!origin && process.env.NODE_ENV !== 'production')
      return callback(null, true);
    // Vérifie que l'origine est dans la whitelist
    if (allowedOrigins.includes(origin))
      return callback(null, true);
    callback(new Error(`CORS bloqué pour l'origine : ${origin}`));
  },
  credentials: true, // Autorise l'envoi des cookies HTTP-only
}));

```

Rate Limiting — Protection contre les attaques par force brute

```

// backend/server.js - Plusieurs niveaux de rate limiting

// 1. Limite globale : 100 requêtes / 15 min par IP
const globalLimiter = rateLimit({ windowMs: 15*60*1000, max: 100 });
app.use('/api', globalLimiter);

// 2. Limite stricte sur le login : 10 tentatives / 15 min
// → Protège contre les attaques brute-force sur les comptes admin
const loginLimiter = rateLimit({ windowMs: 15*60*1000, max: 10 });
app.use('/api/admins/auth/login', loginLimiter);

// 3. Limite sur la soumission de films : 5 / heure
// → Empêche le spam de soumissions automatisées
const submitLimiter = rateLimit({ windowMs: 60*60*1000, max: 5 });
app.use('/api/movies/submit', submitLimiter);

// 4. Limite sur le formulaire de contact : 10 / heure
const contactLimiter = rateLimit({ windowMs: 60*60*1000, max: 10 });
app.use('/api/contact', contactLimiter);

```

Authentification JWT et hachage des mots de passe

L'authentification des administrateurs repose sur JSON Web Tokens (JWT) stockés dans des cookies HTTP-only, ce qui protège contre les attaques XSS. Les mots de passe sont hachés avec bcrypt (coût 10) avant stockage :

```

// Exemple de connexion admin (simplifié)
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');

exports.login = async (req, res, next) => {
  const { email, password } = req.body;

  // 1. Récupérer l'admin par email
  const admin = await adminService.findByEmail(email);
  if (!admin) return res.status(401).json({ error: 'Identifiants invalides' });

  // 2. Comparer le mot de passe avec le hash stocké
  const match = await bcrypt.compare(password, admin.password);
  if (!match) return res.status(401).json({ error: 'Identifiants invalides' });

```



```
// 3. Générer le JWT
const token = jwt.sign(
  { id: admin.id, role: admin.role },
  process.env.JWT_SECRET,
  { expiresIn: process.env.JWT_EXPIRES_IN }
);

// 4. Stocker le token dans un cookie HTTP-only (inaccessible en JS)
res.cookie('token', token, {
  httpOnly: true, // Protection XSS
  secure: process.env.NODE_ENV === 'production', // HTTPS en prod
  sameSite: 'strict', // Protection CSRF
  maxAge: 24 * 60 * 60 * 1000, // 24 heures
});

res.json({ success: true, role: admin.role });
};
```

Protection contre les injections SQL

Toutes les requêtes SQL utilisent des requêtes paramétrées (préparées) via le driver mysql2. Les valeurs fournies par l'utilisateur ne sont jamais interpolées directement dans la chaîne SQL, ce qui neutralise les attaques par injection SQL :

```
// Bonne pratique - requête paramétrée avec placeholders ?
// Les valeurs sont transmises séparément et échappées par le driver
const [rows] = await pool.query(
  'SELECT * FROM admins WHERE email = ? AND role = ?',
  [email, role] // Jamais interpolé directement dans la chaîne
);

// À NE PAS FAIRE - vulnérable à l'injection SQL :
// await pool.query(`SELECT * FROM admins WHERE email = '${email}'`);
```

Job YouTube en arrière-plan

Le back-end intègre un worker (youtubeStatusWorker) qui tourne en arrière-plan pour synchroniser le statut des vidéos YouTube uploadées. Ce job interroge périodiquement l'API YouTube pour vérifier si les vidéos sont disponibles et met à jour le statut en base de données.

[CAPTURE : Screenshot du back-office : galerie des films avec système de flagging vert/jaune/rouge]

[CAPTURE : Screenshot du dashboard admin avec les statistiques]

3. Projet artisan-ai — Assistant WhatsApp IA pour artisans

3.1 Cahier des charges et expression des besoins

3.1.1 Contexte et objectif

artisan-ai est un projet personnel que j'ai conçu et développé seul, partant d'un constat simple : les artisans (plombiers, électriciens, menuisiers, peintres...) reçoivent de nombreuses demandes de clients via WhatsApp, mais n'ont pas toujours le temps d'y répondre immédiatement. Des réponses tardives signifient des clients perdus.

L'objectif est de fournir aux artisans un assistant IA capable de répondre automatiquement à leurs clients WhatsApp, en se faisant passer pour l'artisan lui-même : réponse professionnelle, naturelle, basée sur le profil et les services de l'artisan. L'artisan garde la main via un tableau de bord web pour consulter les conversations et répondre manuellement si nécessaire.

3.1.2 Fonctionnalités cibles

- Inscription et connexion des artisans (email + mot de passe)
- Configuration du profil (nom, nom d'entreprise, bio, services, numéro WhatsApp)
- Réception automatique des messages WhatsApp clients via Twilio
- Réponse IA automatique (Claude d'Anthropic) basée sur le profil de l'artisan
- Dashboard web : liste des conversations, détail des messages
- Système de notifications pour les nouveaux messages
- Réponse manuelle de l'artisan depuis le dashboard
- Système d'abonnement mensuel (Starter 19€, Pro 49€, Équipe 99€) via Stripe
- Portail de facturation Stripe pour gérer l'abonnement

3.1.3 Contraintes techniques

- Application web moderne, responsive, accessible depuis mobile
- Sécurité maximale : authentification robuste, validation des webhooks
- Architecture Next.js App Router pour les performances (Server Components)
- Base de données managée via Supabase (PostgreSQL + Row Level Security)
- Intégration Twilio pour la réception et l'envoi de messages WhatsApp
- Intégration Stripe pour la gestion des abonnements SaaS

[CAPTURE : Captures des pages : landing page, login, dashboard, conversations, profil, billing]

3.2 Gestion de projet

3.2.1 Développement solo et itératif

artisan-ai est un projet solo que j'ai développé en suivant une approche itérative. Sans les contraintes de coordination d'une équipe, j'ai pu avancer fonctionnalité par fonctionnalité, en commençant par le socle technique (authentification, BDD) avant d'ajouter les fonctionnalités métier (WhatsApp, IA, Stripe).

Ordre de développement adopté :

- Sprint 1 : Mise en place du projet Next.js, configuration Supabase, authentification complète
- Sprint 2 : Dashboard, profil utilisateur, Server Actions
- Sprint 3 : Intégration Twilio WhatsApp (webhook, réception et envoi de messages)
- Sprint 4 : Intégration Claude AI pour les réponses automatiques
- Sprint 5 : Système de billing Stripe (abonnements, webhook, portail)
- Sprint 6 : Notifications, polissage UI, tests

3.2.2 Gestion des versions

Git a été utilisé avec une convention de branches feature-based. Chaque fonctionnalité majeure (auth, whatsapp, stripe, ai) était développée sur sa propre branche avant d'être mergée dans main.

3.3 Spécifications techniques

3.3.1 Architecture Next.js App Router

artisan-ai exploite le paradigme App Router de Next.js 14, qui permet de mixer Server Components (rendus côté serveur, sans JavaScript client) et Client Components (interactifs, côté navigateur). Cette architecture offre de meilleures performances initiales de chargement et simplifie l'accès aux données côté serveur.

```
Structure du projet artisan-ai :
src/
├── app/
│   ├── (auth)/                # Groupe de routes auth (layout sans sidebar)
│   │   ├── login/             # Page de connexion
│   │   └── register/          # Page d'inscription
│   ├── (dashboard)/           # Groupe de routes protégées
│   │   ├── dashboard/         # Tableau de bord principal
│   │   ├── conversations/     # Liste et détail des conversations
│   │   ├── profile/           # Gestion du profil artisan
│   │   └── billing/           # Gestion de l'abonnement Stripe
│   ├── api/
│   │   ├── billing/           # Routes API Stripe
│   │   └── webhooks/
│   │       └── whatsapp/      # Webhook Twilio entrant
│   └── auth/confirm/          # Callback confirmation email
└── components/
```

├── dashboard/	# Composants du dashboard
├── ui/	# Composants shadcn/ui
├── lib/	
│ ├── actions/	# Server Actions (auth, profile, messages)
│ ├── supabase/	# Clients Supabase (client, server, admin)
│ ├── stripe.ts	# Instance Stripe
│ └── plans.ts	# Définition des plans tarifaires
├── middleware.ts	# Protection des routes (Next.js middleware)
└── types/database.ts	# Types TypeScript de la BDD Supabase

3.3.2 Justification des choix technologiques

Next.js 14 avec App Router

Next.js est le framework React le plus utilisé pour les applications full-stack. L'App Router apporte les React Server Components, permettant de récupérer les données directement dans les composants sans exposer de logique sensible côté client. Cela simplifie considérablement l'architecture par rapport à des allers-retours client/serveur classiques.

TypeScript

TypeScript a été choisi pour sa sécurité de typage, particulièrement précieuse dans un projet solo où il n'y a pas d'autre développeur pour relire le code. Les types générés depuis le schéma Supabase permettent de détecter les erreurs à la compilation plutôt qu'à l'exécution.

Supabase (PostgreSQL + Auth + RLS)

Supabase est un Backend-as-a-Service open source basé sur PostgreSQL. Il fournit l'authentification (email/password, OAuth), la base de données PostgreSQL avec Row Level Security (RLS — politique d'accès aux données par rôle), et une API auto-générée. C'est une alternative puissante à Firebase, avec l'avantage d'utiliser SQL standard.

Stripe (abonnements SaaS)

Stripe est la référence pour le paiement en ligne. Son API robuste permet de gérer des abonnements récurrents, des essais gratuits, des changements de plan et les webhooks de paiement. Le portail Stripe permet aux clients de gérer eux-mêmes leur abonnement sans que j'aie à développer cette interface.

Twilio (WhatsApp)

Twilio fournit une API permettant d'envoyer et recevoir des messages WhatsApp via un webhook HTTP. Quand un client envoie un message WhatsApp à l'artisan, Twilio transmet ce message à mon API Next.js via une requête POST signée.

Claude API (Anthropic)

L'API Claude (claude-sonnet-4-6) génère les réponses automatiques en se basant sur le profil de l'artisan et l'historique de la conversation. Le modèle est instruit de répondre naturellement, sans se présenter comme une IA, dans le style de l'artisan.

shadcn/ui (composants UI)

shadcn/ui est une collection de composants React accessibles et personnalisables, basés sur Radix UI et stylisés avec Tailwind CSS. Il accélère considérablement le développement d'une interface professionnelle sans sacrifier la personnalisation.

3.4 AT1 — Développement de la partie front-end

3.4.1 Maquettage et charte graphique

artisan-ai utilise une interface sobre et professionnelle, conçue pour être utilisée quotidiennement par des artisans qui n'ont pas forcément une grande affinité avec les outils numériques. Les choix UI privilégient la clarté et l'efficacité :

- Palette : blanc cassé comme fond, bleu/indigo comme couleur primaire, gris neutres pour les surfaces
- Composants shadcn/ui : cohérence visuelle garantie, accessibilité native (ARIA)
- Sidebar fixe sur desktop, repliable sur mobile (hook use-mobile)
- Icônes Lucide React pour une iconographie unifiée

[CAPTURE : Screenshot de la page de connexion artisan-ai]

[CAPTURE : Screenshot du dashboard principal avec la sidebar]

[CAPTURE : Screenshot de la page des conversations (liste + détail)]

[CAPTURE : Screenshot de la page de profil artisan]

[CAPTURE : Screenshot de la page de billing avec les plans Stripe]

3.4.2 Interfaces statiques — Composants shadcn/ui

L'ensemble des composants UI de base (boutons, inputs, cards, tables, dialogs...) sont fournis par shadcn/ui et personnalisés selon la charte graphique du projet. La sidebar de navigation est un exemple de composant complexe avec état de collapse :

```
// src/components/dashboard/sidebar.tsx (extrait)
// Sidebar de navigation responsive
'use client'

import { usePathname } from 'next/navigation'
import {
  Sidebar, SidebarContent, SidebarMenu,
  SidebarMenuItem, SidebarMenuButton
```

```

} from '@components/ui/sidebar'
import { MessageSquare, User, CreditCard, LayoutDashboard } from 'lucide-react'

const navItems = [
  { href: '/dashboard', icon: LayoutDashboard, label: 'Dashboard' },
  { href: '/conversations', icon: MessageSquare, label: 'Conversations' },
  { href: '/profile', icon: User, label: 'Profil' },
  { href: '/billing', icon: CreditCard, label: 'Abonnement' },
]

export function AppSidebar() {
  const pathname = usePathname()
  return (
    <Sidebar>
      <SidebarContent>
        <SidebarMenu>
          {navItems.map(item => (
            <SidebarMenuItem key={item.href}>
              <SidebarMenuButton
                asChild
                isActive={pathname.startsWith(item.href)}
              >
                <a href={item.href}>
                  <item.icon />
                  <span>{item.label}</span>
                </a>
              </SidebarMenuButton>
            </SidebarMenuItem>
          ))}
        </SidebarMenu>
      </SidebarContent>
    </Sidebar>
  )
}

```

3.4.3 Interfaces dynamiques — Server Components et Server Actions

Next.js App Router permet d'écrire des composants qui s'exécutent côté serveur, accédant directement à la base de données sans exposer de logique sensible au client. Les Server Actions remplacent les routes API classiques pour les mutations de données :

```

// src/lib/actions/auth.ts - Server Actions pour l'authentification
'use server'

import { redirect } from 'next/navigation'
import { createClient } from '@lib/supabase/server'

export async function login(formData: FormData) {
  const supabase = await createClient()

  const { error } = await supabase.auth.signInWithPassword({
    email: formData.get('email') as string,
    password: formData.get('password') as string,
  })
}

```

```

    if (error) return { error: error.message }
    redirect('/dashboard') // Redirection serveur sécurisée
  }

export async function register(formData: FormData) {
  const supabase = await createClient()

  const { error } = await supabase.auth.signUp({
    email: formData.get('email') as string,
    password: formData.get('password') as string,
    options: {
      data: { full_name: formData.get('full_name') as string },
    },
  })

  if (error) return { error: error.message }
  redirect('/dashboard')
}

```

Affichage des conversations en temps quasi-réel

La page des conversations récupère les données côté serveur (Server Component) pour le rendu initial, puis utilise un composant client pour la mise à jour des messages :

```

// src/app/(dashboard)/conversations/[id]/page.tsx (simplifié)
// Server Component : récupère la conversation et les messages
import { createClient } from '@lib/supabase/server'
import { ReplyBox } from '@components/dashboard/reply-box'

export default async function ConversationPage({
  params
}: { params: { id: string } }) {
  const supabase = await createClient()

  // Récupération des messages depuis le serveur
  const { data: messages } = await supabase
    .from('messages')
    .select('*')
    .eq('conversation_id', params.id)
    .order('created_at', { ascending: true })

  return (
    <div>
      { /* Affichage des messages */ }
      { messages?.map(msg => (
        <div key={msg.id}
          className={msg.from_type === 'client' ? 'text-left' : 'text-right'}
        >
          <span>{msg.content}</span>
        </div>
      )) }
      { /* Client Component pour envoyer des réponses */ }
      <ReplyBox conversationId={params.id} />
    </div>
  )
}

```

```
)  
}
```

3.4.4 Démarche de sécurité côté front-end

Protection des routes — Middleware Next.js

Le middleware Next.js intercepte chaque requête avant qu'elle n'atteigne une page ou une route API. Il vérifie si l'utilisateur est authentifié et redirige vers /login si une route protégée est accédée sans session active :

```
// src/lib/supabase/middleware.ts  
// Exécuté sur le serveur Edge pour chaque requête  
export async function updateSession(request: NextRequest) {  
  const supabase = createServerClient(/* config Supabase */)  
  
  // Vérification de la session utilisateur  
  const { data: { user } } = await supabase.auth.getUser()  
  
  // Routes protégées — redirige vers /login si pas connecté  
  const protectedPaths = ['/dashboard', '/conversations', '/profile',  
    '/billing']  
  const isProtected = protectedPaths.some(  
    p => request.nextUrl.pathname.startsWith(p)  
  )  
  
  if (isProtected && !user) {  
    const url = request.nextUrl.clone()  
    url.pathname = '/login'  
    return NextResponse.redirect(url) // Redirection sécurisée  
  }  
  
  // Si déjà connecté sur login/register → redirige vers dashboard  
  const authPaths = ['/login', '/register']  
  if (authPaths.some(p => request.nextUrl.pathname.startsWith(p)) && user) {  
    const url = request.nextUrl.clone()  
    url.pathname = '/dashboard'  
    return NextResponse.redirect(url)  
  }  
  
  return supabaseResponse  
}
```

3.5 AT2 — Développement de la partie back-end

3.5.1 Conception de la base de données

Schéma PostgreSQL (Supabase)

La base de données artisan-ai s'articule autour de cinq tables principales, définies en TypeScript pour un typage strict dans toute l'application :


```

-- Table profiles : profil public de chaque artisan
CREATE TABLE profiles (
  id          UUID PRIMARY KEY REFERENCES auth.users(id),
  full_name   TEXT,
  business_name TEXT,
  phone       TEXT,
  whatsapp_number TEXT,    -- Numéro utilisé par Twilio
  bio         TEXT,        -- Décrit l'activité à l'IA
  services    TEXT[],      -- Tableau de services proposés
  created_at  TIMESTAMPTZ DEFAULT now(),
  updated_at  TIMESTAMPTZ DEFAULT now()
);

-- Table subscriptions : abonnements Stripe
CREATE TABLE subscriptions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID REFERENCES auth.users(id) UNIQUE,
  stripe_customer_id TEXT,
  stripe_subscription_id TEXT,
  plan        TEXT CHECK (plan IN ('starter','pro','team')),
  status       TEXT CHECK (status IN
('active','canceled','past_due','trialing')),
  current_period_end TIMESTAMPTZ
);

-- Table conversations : fil de discussion client/artisan
CREATE TABLE conversations (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  artisan_id  UUID REFERENCES auth.users(id),
  client_phone TEXT NOT NULL,    -- Numéro WhatsApp du client
  client_name TEXT,              -- Nom WhatsApp du client
  status       TEXT DEFAULT 'open' CHECK (status IN ('open','closed')),
  last_message_at TIMESTAMPTZ
);

-- Table messages : messages individuels
CREATE TABLE messages (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  conversation_id UUID REFERENCES conversations(id),
  from_type    TEXT CHECK (from_type IN ('client','ai','artisan')),
  content       TEXT NOT NULL,
  whatsapp_sid TEXT -- ID du message Twilio pour le suivi
);

-- Table notifications : alertes pour l'artisan
CREATE TABLE notifications (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  artisan_id  UUID REFERENCES auth.users(id),
  type        TEXT CHECK (type IN ('new_message','new_conversation')),
  message     TEXT NOT NULL,
  read        BOOLEAN DEFAULT false
);

```

[CAPTURE : Diagramme entité-relation de la BDD artisan-ai (profiles, subscriptions, conversations, messages, notifications)]

3.5.2 Composants d'accès aux données — Clients Supabase

Supabase fournit trois clients selon le contexte d'exécution, chacun avec des permissions différentes :

```
// src/lib/supabase/server.ts
// Client serveur : accès avec les cookies de session de l'utilisateur
// Utilisé dans les Server Components et Server Actions
import { createServerClient } from '@supabase/ssr'
import { cookies } from 'next/headers'

export async function createClient() {
  const cookieStore = await cookies()
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (cs) => cs.forEach(({ name, value, options }) =>
          cookieStore.set(name, value, options)
        ),
      },
    }
  )
}

// src/lib/supabase/admin.ts
// Client admin : utilise la SERVICE_ROLE_KEY, bypass le RLS
// Utilisé UNIQUEMENT dans les webhooks serveur (Twilio, Stripe)
import { createClient } from '@supabase/supabase-js'

export function createAdminClient() {
  return createClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.SUPABASE_SERVICE_ROLE_KEY!, // Clé secrète — jamais exposée
  )
}
```

3.5.3 Routes API — Webhooks Stripe et Twilio

Webhook WhatsApp Twilio

Le webhook WhatsApp est la pièce centrale d'artisan-ai. Quand un client envoie un message WhatsApp à l'artisan, Twilio envoie une requête POST à cette route. Voici le flux complet :

```
// src/app/api/webhooks/whatsapp/route.ts (flux simplifié)
// 1. Valider la signature Twilio (sécurité)
// 2. Parser le message entrant
// 3. Trouver l'artisan via son numéro WhatsApp
// 4. Créer ou retrouver la conversation
// 5. Stocker le message client
// 6. Générer la réponse IA via Claude
// 7. Retourner la réponse en TwiML (format XML Twilio)

export async function POST(request: NextRequest) {
```

```

const rawBody = await request.text()

// Sécurité : en production, rejeter les requêtes non signées
if (process.env.NODE_ENV === 'production') {
  if (!validateTwilioSignature(request, rawBody))
    return new NextResponse('Forbidden', { status: 403 })
}

// ... (traitement, réponse IA, insertion BDD)

return twiml(aiReply) // Réponse XML pour Twilio
}

```

Webhook Stripe

Le webhook Stripe met à jour la base de données lorsqu'un événement de paiement se produit (abonnement créé, renouvelé, annulé, paiement échoué) :

```

// src/app/api/billing/webhook/route.ts (extrait)
export async function POST(request: NextRequest) {
  const body = await request.text()
  const signature = request.headers.get('stripe-signature')!

  // Validation de la signature Stripe pour rejeter les faux webhooks
  const event = stripe.webhooks.constructEvent(
    body, signature, process.env.STRIPE_WEBHOOK_SECRET!
  )

  switch (event.type) {
    case 'customer.subscription.created':
    case 'customer.subscription.updated':
      // Mettre à jour le plan et statut en BDD
      await updateSubscription(event.data.object)
      break
    case 'customer.subscription.deleted':
      await cancelSubscription(event.data.object)
      break
  }

  return NextResponse.json({ received: true })
}

```

3.5.4 Démarche de sécurité côté back-end

Validation de la signature Twilio (HMAC SHA-1)

Twilio signe chaque requête webhook avec un HMAC SHA-1 utilisant le Auth Token du compte. Cette validation est critique : sans elle, n'importe qui pourrait envoyer de fausses requêtes à notre endpoint et injecter des messages dans les conversations.

```

// src/app/api/webhooks/whatsapp/route.ts
// Validation de la signature Twilio - protection contre les faux webhooks
function validateTwilioSignature(request: NextRequest, body: string): boolean {
  const authToken = process.env.TWILIO_AUTH_TOKEN

```

```

const signature = request.headers.get('x-twilio-signature')

if (!authToken || !signature) return false

const url = process.env.TWILIO_WEBHOOK_URL ?? request.url

// Reconstitution de la chaîne signée (URL + params triés par clé)
const params = new URLSearchParams(body)
const sortedKeys = [...params.keys()].sort()
const paramString = sortedKeys
  .map(k => `${k}${params.get(k)}`)
  .join('&')

// Calcul du HMAC SHA-1 attendu
const expected = crypto
  .createHmac('sha1', authToken)
  .update(url + paramString)
  .digest('base64')

// Comparaison sécurisée (protection contre les timing attacks)
return crypto.timingSafeEqual(
  Buffer.from(signature),
  Buffer.from(expected)
)
}

```

Note importante : l'utilisation de `crypto.timingSafeEqual()` au lieu d'un simple `===` est délibérée. Une comparaison de chaînes standard prend plus ou moins de temps selon le nombre de caractères identiques, ce qui permettrait à un attaquant de deviner progressivement la signature valide via des timing attacks. `timingSafeEqual()` prend toujours le même temps quelle que soit la comparaison.

Row Level Security (RLS) de Supabase

Supabase permet de définir des politiques de sécurité directement au niveau de la base de données PostgreSQL. Ces politiques garantissent qu'un artisan ne peut accéder qu'à ses propres données, même si l'application avait un bug :

```

-- Politique RLS : un artisan ne voit que ses propres conversations
CREATE POLICY 'artisan_sees_own_conversations'
  ON conversations FOR SELECT
  USING (auth.uid() = artisan_id);

-- Politique RLS : un artisan ne voit que ses propres messages
CREATE POLICY 'artisan_sees_own_messages'
  ON messages FOR SELECT
  USING (
    conversation_id IN (
      SELECT id FROM conversations WHERE artisan_id = auth.uid()
    )
  );

```

Authentification sécurisée avec Supabase Auth

Supabase Auth gère l'authentification de bout en bout : hachage des mots de passe (bcrypt), sessions sécurisées via cookies httpOnly, confirmation d'email avec token signé, et protection contre les attaques par force brute.

```
// src/app/auth/confirm/route.ts
// Endpoint de confirmation d'email après inscription
export async function GET(request: NextRequest) {
  const { searchParams } = new URL(request.url)
  const token_hash = searchParams.get('token_hash')
  const type = searchParams.get('type') as EmailOtpType | null

  if (token_hash && type) {
    const supabase = await createClient()
    // Vérification du token signé envoyé par email
    const { error } = await supabase.auth.verifyOtp({ type, token_hash })
    if (!error) {
      return NextResponse.redirect(new URL('/dashboard', request.url))
    }
  }
  return NextResponse.redirect(new URL('/login?error=invalid_token', request.url))
}
```

Plans et tarifs Stripe

La définition des plans d'abonnement est centralisée dans un fichier partagé entre le client et le serveur, garantissant la cohérence des prix affichés et facturés :

```
// src/lib/plans.ts - Plans tarifaires artisan-ai
export const PLANS = {
  starter: {
    name: 'Starter', price: '19€',
    features: ['50 réponses IA / mois', '1 numéro WhatsApp',
      'Dashboard web', 'Notifications par email'],
  },
  pro: {
    name: 'Pro', price: '49€',
    features: ['Réponses IA illimitées', '1 numéro WhatsApp',
      'Dashboard web', 'Support prioritaire'],
  },
  team: {
    name: 'Équipe', price: '99€',
    features: ["Tout ce qui est dans Pro", "Jusqu'à 5 artisans",
      '5 numéros WhatsApp', 'Statistiques avancées'],
  },
} as const
```

[CAPTURE : Screenshot de la page billing avec les 3 plans Starter / Pro / Équipe]

[CAPTURE : Screenshot du portail Stripe pour la gestion de l'abonnement]

4. Veille technologique et de sécurité

4.1 Importance de la veille en développement web

Le développement web est un domaine en évolution constante. Les frameworks se mettent à jour, de nouvelles vulnérabilités sont découvertes, et les meilleures pratiques évoluent rapidement. En tant que développeur, maintenir une veille active est une nécessité professionnelle, non un luxe.

Au cours de ma formation et de mes projets, j'ai mis en place une routine de veille qui me permet de rester informé sur les évolutions technologiques et les enjeux de sécurité.

4.2 Sources et méthodes

4.2.1 Documentation officielle

Ma première source de référence reste toujours la documentation officielle. Pour les technologies utilisées dans mes projets :

- MDN Web Docs (developer.mozilla.org) : référence HTML, CSS, JavaScript — toujours à jour
- React.dev : documentation officielle React avec les nouveautés (hooks, Server Components)
- Next.js Documentation (nextjs.org/docs) : App Router, Server Actions, optimisations
- Supabase Docs : authentification, RLS, PostgreSQL
- Stripe Docs : webhooks, SDK, gestion des abonnements
- Twilio Docs : WhatsApp Business API, validation des signatures

4.2.2 Communauté et blogs techniques

- Dev.to et Medium : articles de blog écrits par des développeurs, cas pratiques, retours d'expérience
- GitHub Changelog et Release Notes : je suis les dépôts des bibliothèques utilisées (React, Next.js, Supabase) pour anticiper les changements de version majeurs
- Reddit ([r/webdev](https://www.reddit.com/r/webdev), [r/reactjs](https://www.reddit.com/r/reactjs)) : discussions de la communauté, questions/réponses, partage d'expériences

4.2.3 Ressources vidéo

- YouTube : chaînes comme Fireship, Kevin Powell, Traversy Media, Web Dev Simplified pour des explications visuelles de nouveaux concepts
- Conférences enregistrées (React Summit, Next.js Conf) pour suivre l'évolution des frameworks

4.2.4 Veille sécurité

La sécurité est un aspect non négociable du développement web. Ma veille sécurité s'appuie sur :

- OWASP Top 10 (owasp.org) : les dix vulnérabilités web les plus critiques, avec des guides de prévention détaillés. J'ai utilisé ce référentiel pour structurer ma démarche de sécurité sur marsAI (injections SQL, XSS, CSRF, broken authentication...)
- CVE (Common Vulnerabilities and Exposures) : alertes sur les vulnérabilités connues des bibliothèques utilisées
- npm audit : outil intégré à npm pour détecter les dépendances vulnérables dans les projets Node.js
- Bulletins de sécurité Supabase et Stripe : emails d'alerte des plateformes utilisées

4.3 Application concrète de la veille

Ma veille a eu des impacts concrets sur mes projets :

- Helmet sur marsAI : découvert via la documentation OWASP sur les headers HTTP de sécurité. Helmet configure automatiquement Content-Security-Policy, X-Frame-Options, et d'autres headers protecteurs.
- crypto.timingSafeEqual sur artisan-ai : découvert lors de la lecture de la documentation Node.js sur la cryptographie. L'utilisation systématique de comparaisons en temps constant protège contre les attaques de type timing attack.
- Row Level Security Supabase : appris via la documentation Supabase et des articles sur l'architecture multi-tenant, pour garantir l'isolation des données entre artisans.
- Cookies HTTP-only pour JWT : best practice apprise via des articles sur les vulnérabilités XSS, qui soulignent que localStorage est accessible au JavaScript malveillant, contrairement aux cookies httpOnly.

5. Situation professionnelle ayant nécessité une recherche en anglais

5.1 Contexte

Dans le cadre du développement d'artisan-ai, j'ai dû intégrer l'API Twilio pour la réception et l'envoi de messages WhatsApp. La documentation Twilio est exclusivement disponible en anglais, ce qui m'a obligé à travailler en profondeur avec des ressources anglophones : documentation officielle, guides d'intégration, exemples de code, et forum de support.

5.2 Problème rencontré

La partie la plus complexe a été la validation de la signature Twilio. Twilio signe chaque requête webhook avec un HMAC SHA-1 pour permettre au développeur de vérifier que la requête provient bien de Twilio et non d'un acteur malveillant. J'ai dû comprendre et implémenter cet algorithme de validation en me basant uniquement sur la documentation anglophone.

5.3 Recherche en anglais

J'ai consulté les ressources suivantes, toutes en anglais :

- Twilio official documentation — Validate that HTTP requests to your Twilio application are from Twilio
- Node.js documentation — `crypto.timingSafeEqual()` method
- Twilio community forum — posts discussing HMAC validation edge cases
- GitHub issues on the Twilio Node.js SDK repository

5.4 Présentation en anglais de la solution

Below is a summary in English of the technical solution I implemented to validate Twilio webhook signatures in artisan-ai.

When Twilio sends a webhook to my endpoint, it includes an X-Twilio-Signature header containing a Base64-encoded HMAC-SHA1 hash. To validate this signature, I reconstruct the same hash server-side by combining the full webhook URL with the sorted POST parameters. I then use Node.js's `crypto.createHmac('sha1', authToken)` to compute the expected signature and compare it against the received one using `crypto.timingSafeEqual()`. This constant-time comparison is critical to prevent timing attacks — a vulnerability where an attacker could guess the valid signature character by character by measuring the time difference between comparisons. In production, any request that fails this validation is immediately rejected with a 403 Forbidden

response, ensuring that only genuine Twilio webhooks can trigger AI responses and write to the database.

5.5 Extrait de code commenté en anglais

```
// Twilio Signature Validation - Node.js implementation
// Source: https://www.twilio.com/docs/usage/security

function validateTwilioSignature(request: NextRequest, body: string): boolean {
  const authToken = process.env.TWILIO_AUTH_TOKEN // Your Twilio Auth Token
  const signature = request.headers.get('x-twilio-signature') // Twilio's
  signature

  if (!authToken || !signature) return false

  const url = process.env.TWILIO_WEBHOOK_URL ?? request.url

  // Step 1: Sort all POST parameters alphabetically by key name
  const params = new URLSearchParams(body)
  const sortedKeys = [...params.keys()].sort()
  // Step 2: Append each param (key + value) to the URL string
  const paramString = sortedKeys
    .map(k => `${k}${params.get(k)}`)
    .join('&')

  // Step 3: Sign the string with HMAC-SHA1 using your Auth Token
  const expected = crypto
    .createHmac('sha1', authToken)
    .update(url + paramString)
    .digest('base64')

  // Step 4: Compare using timingSafeEqual to prevent timing attacks
  // (constant-time comparison - takes the same time regardless of differences)
  return crypto.timingSafeEqual(
    Buffer.from(signature),
    Buffer.from(expected)
  )
}
```

5.6 Compétences linguistiques mobilisées

Cette expérience m'a mobilisé les compétences suivantes en anglais technique :

- Lecture et compréhension de documentation technique dense (algorithmes cryptographiques, protocoles HTTP)
- Déchiffrement de code d'exemple en anglais et adaptation à mon architecture (TypeScript, Next.js)
- Lecture des discussions sur forums et GitHub Issues pour trouver des solutions à des cas particuliers

- Compréhension des messages d'erreur en anglais retournés par les API Twilio et Node.js

Cette capacité à lire et exploiter des ressources en anglais est aujourd'hui indispensable pour tout développeur web, l'écrasante majorité de la documentation, des bibliothèques et des ressources de la communauté étant rédigée dans cette langue.

6. Conclusion

6.1 Bilan des compétences acquises

La réalisation de ces deux projets m'a permis de développer et de démontrer l'ensemble des compétences définies par le référentiel DWWM.

AT1 — Développement front-end

Sur marsAI, j'ai acquis la maîtrise de React 19 dans un contexte d'équipe : conception de composants réutilisables, gestion d'état global via Context API, routing avec React Router, internationalisation (i18n), et intégration avec une API REST. J'ai également appris à structurer un projet front-end de taille réelle avec une architecture modulaire claire.

Sur artisan-ai, j'ai découvert et maîtrisé le paradigme App Router de Next.js 14, avec ses Server Components, ses Server Actions et son middleware Edge. Cette approche représente l'état de l'art du développement React en 2025/2026 et m'a ouvert à des architectures plus performantes et plus sécurisées.

AT2 — Développement back-end

marsAI m'a formé à la conception et l'implémentation d'une API REST complète avec Express.js : architecture MVC, pool de connexions MariaDB, requêtes SQL paramétrées, authentification JWT, et intégration de multiples services tiers (S3, Brevo, YouTube). J'ai également été confronté aux enjeux de déploiement d'une application full-stack avec Docker.

artisan-ai m'a familiarisé avec les approches modernes du back-end web : Supabase comme Backend-as-a-Service, Row Level Security PostgreSQL, Server Actions Next.js, et l'intégration de webhooks sécurisés (Twilio, Stripe). Ce projet m'a aussi permis de travailler pour la première fois avec une API d'intelligence artificielle (Claude d'Anthropic).

Compétences transversales

- Sécurité web : application concrète des recommandations OWASP (XSS, injection SQL, CSRF, authentification sécurisée, hachage bcrypt, cookies HTTP-only, rate limiting, validation des signatures)
- Gestion de projet : expérience de la méthodologie Agile en équipe (marsAI) et en solo (artisan-ai), avec GitHub Projects et stand-ups quotidiens
- Contrôle de version : utilisation avancée de Git (branches, Pull Requests, code review, conventions de commit)
- Anglais technique : lecture et exploitation de documentation technique en anglais, indispensable dans l'écosystème du développement web
- Veille technologique : mise en place d'une routine de veille pour rester à jour sur les évolutions des frameworks et les enjeux de sécurité

6.2 Axes d'amélioration et perspectives

Ces projets m'ont également permis d'identifier des axes d'amélioration que j'entends travailler dans mes prochaines expériences professionnelles :

- Tests automatisés : je n'ai pas suffisamment testé mes applications (tests unitaires, tests d'intégration, tests end-to-end avec Playwright ou Cypress). C'est une pratique que je veux systématiser.
- CI/CD : mettre en place des pipelines d'intégration et de déploiement continus (GitHub Actions) pour automatiser les tests et les déploiements
- Accessibilité (a11y) : bien que shadcn/ui propose des composants accessibles, je n'ai pas testé mes interfaces avec des lecteurs d'écran. C'est un aspect important que je veux approfondir
- Performance et monitoring : intégrer des outils comme Sentry (gestion des erreurs) et des dashboards de métriques pour suivre la santé des applications en production
- Architecture micro-services : pour des projets de plus grande envergure, envisager une décomposition en micro-services indépendants

6.3 Mot final

Ces 16 mois de formation à La Plateforme ont été intenses, stimulants et fondateurs. J'arrive à ce titre professionnel avec deux projets concrets qui témoignent de ma capacité à concevoir, développer et sécuriser des applications web full-stack de A à Z, seul ou en équipe.

Le développement web est un métier que j'exerce avec passion et dans lequel je m'engage à continuer de progresser, en restant attentif aux évolutions technologiques et aux meilleures pratiques de l'industrie.

Amad-Khaly Diallo — Juin 2026